

Práctica 1: Ecuaciones en una variable

8 de abril de 2026

1. Ejercicios

1. Para encontrar una solución positiva a $\tan(x) = 2x$, consideremos $f(x) = \tan(x) - 2x$. Es claro que $f(\frac{1}{2}) < 0$ ya que $\tan(\frac{1}{2}) < \tan(\frac{\pi}{4}) = 1 = 2 \cdot 1$, y se puede ver que $f(\frac{3}{2}) > 0$. Como $\tan(x)$ es continua en $\mathbb{R} \setminus \{\pi \cdot (k + \frac{1}{2}), k \in \mathbb{Z}\}$, también lo será f gracias a la continuidad de $2x$, y lo será en particular en $[\frac{1}{2}, \frac{3}{2}]$. De esta manera, estamos en condiciones de aplicar el método de bisección para aproximar una raíz de f . Luego, para que el error relativo sea menor que 10^{-5} tenemos que:

$$\frac{|e_n|}{|p|} < \frac{\frac{3}{2} - \frac{1}{2}}{2^n} = \frac{1}{3 \cdot 2^{n-1}} < 10^{-5} \iff \log_2\left(\frac{10^5}{3}\right) + 1 < n$$

Usando que $\lceil \log_2(\frac{10^5}{3}) \rceil = 16$, se deduce que para $n > 17$ se obtiene una aproximación tan chica como se quiere.

Con la implementación adjunta al final del documento, se obtiene el resultado $x \approx 1,16556$.

2. Para estimar el valor de $x = \sqrt[3]{2}$, igualando a 0 puedo Obtener $f(x) = x^3 - 2$, consiguiendo así reducir el problema a uno del tipo $f(x) = 0$. Veamos rápidamente que:

- f es continua en todo $\mathbb{R}$, por ser polinómica.
- $f(0) = -2 < 0$.
- $f(2) = 8 - 2 = 6 > 0$.
- f es continua, en particular, sobre $[0, 2]$.

De esta forma, estamos en condiciones de aplicar el método de bisección para hallar una raíz de f . Aplicando 20 iteraciones de éste método, se consigue el valor $x \approx 1,25992107$.

3. Consideremos $f(x) = x^3 + x - 4$. Es fácil ver que f es continua $\forall x \in \mathbb{R}$, en particular sobre $[1, 4]$. Además, $f(1) = -2 < 0$ y $f(4) = 64 > 0$, por lo que estamos en condiciones de aplicar el teorema 2.1.

Podemos asegurar que $|e_n| \leq \frac{3}{2^n}$. Luego, basta pedir $\frac{3}{2^n} \leq 10^{-3}$ para asegurar que el error absoluto es tan chico como se quiere. Despejando n :

$$\frac{3}{2^n} \leq 10^{-3} \iff 3000 \leq 2^n \iff \log_2(3000) \approx 11,55 \leq n$$

Por lo que para 12 iteraciones se consigue una aproximación lo suficientemente buena, una vez ejecutado, el valor obtenido es $x \approx 1,37$.

4. Sea $f(x) = 2 + \cos(e^x - 2) - e^x$. Se puede ver usando desigualdades que $f(1,5) < 0 < f(0,5)$. Además, f es continua sobre $[0,5, 1,5]$, gracias a ser composición y sumas de continuas. De esta forma, los teoremas de bisección son válidos. Para acotar el error por 10^{-3} :

$$|e_n| \leq \frac{1}{2^n} < 10^{-3} \iff \log_2(1000) < n \iff 9 < n$$

De esta forma, se puede aplicar el método 9 veces para obtener $x \approx 1,007$.

5. Dada $f(x) = x^3 - x - 1$, analizaremos las iteraciones de punto fijo $g(x) = x^3 - 1$ y $h(x) = \sqrt[3]{x+1}$.

- a) Para ver cuál es adecuada, debemos ver que la misma cumpla con los requisitos impuestos por el teorema de punto fijo. Veamos primero cuándo $|g'(x)| < 1$:

$$\begin{aligned} 0 &\leq 3x^2 < 1 \\ 0 &\leq x^2 < \frac{1}{3} \\ -\frac{1}{\sqrt{3}} &< x < \frac{1}{\sqrt{3}} \end{aligned}$$

Obteniendo así el intervalo $(-\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}})$. Analizando qué pasa con las imágenes de g :

$$\begin{aligned} -\frac{1}{\sqrt{3}} &< x < \frac{1}{\sqrt{3}} \\ -\frac{1}{\sqrt{27}} &< x^3 < \frac{1}{\sqrt{27}} \\ -\frac{1}{\sqrt{27}} - 1 &< -1 < x^3 - 1 < \frac{1}{\sqrt{27}} - 1 \end{aligned}$$

Deduciendo que g no mapea el intervalo en sí mismo (esto se ve gracias a la cota inferior obtenida), por lo que g no es óptima para la iteración de punto fijo.

Analicemos ahora h . Trabajando su derivada:

$$\begin{aligned}
0 &\leq \frac{1}{3\sqrt[3]{(x+1)^2}} < 1 \\
0 &\leq \frac{1}{\sqrt[3]{(x+1)^2}} < 3 \\
0 &\leq \frac{1}{(x+1)^2} < 27 \\
-\sqrt{27} &< \frac{1}{x+1} < \sqrt{27}
\end{aligned}$$

Esto último ocurre si $x+1 > \frac{1}{\sqrt{27}}$, o bien $x+1 < -\frac{1}{\sqrt{27}}$. Comprobando rápidamente:

- Si $x+1 < -\frac{1}{\sqrt{27}}$, entonces $\sqrt[3]{x+1} < -\frac{1}{\sqrt{3}}$. Esto es un problema ya que la existencia de imágenes en $(-1, -\frac{1}{3})$ muestra que el intervalo no se mapea en sí mismo.
- Sin embargo, si $x+1 > \frac{1}{\sqrt{27}}$, entonces $\sqrt[3]{x+1} > \frac{1}{\sqrt{3}}$. De esta forma obtengo una cota inferior sobre la cual puedo construir un intervalo que se mapee en sí mismo. En particular, tomando $[0, 2]$, obtengo que $0 < x < 2 \iff 1 < x+1 < 3 \iff 1 < \sqrt[3]{x+1} < \sqrt[3]{3} < 2$, consiguiendo así el intervalo que quería.

La continuidad de h es trivial por ser composición de continuas, lo cual vale en todo intervalo. de esta forma, concluyo que h es la iteración de punto fijo adecuada, en el intervalo $[0, 2]$.

- b) Analizando $g''(x) = -\frac{2}{9\sqrt[3]{(x+1)^5}}$ se puede ver que g' tiene como máximo $g'(0) = \frac{1}{3}$. Luego, basta tomar $k = \frac{1}{2}$ para acotar el error. Como sigue:

$$\begin{aligned}
|e_n| &\leq k^n \max\{p_0 - a, b - p_0\} < 10^{-3} \\
10^3 \max\{p_0 - a, b - p_0\} &< \frac{1}{k^n} \\
\frac{1}{\log_2(\frac{1}{k})} \cdot 10^3 \max\{p_0 - a, b - p_0\} &< n
\end{aligned}$$

Usando el intervalo inicial $[0, 2]$ con $p_0 = 1$ y $k = \frac{1}{2}$, se tiene que:

$$10^3 < n$$

Ejecutando el algoritmo con esos parámetros se obtiene $x \approx 1,32$.

6. a) Se vera

7. Analizando la expresión de $s(t) = s_0 - \frac{mg}{k}t + \frac{m^2g}{k^2}(1 - e^{\frac{-kt}{m}})$

2. Implementaciones

- Algoritmo de bisección

```
# Implementación del método de bisección
def bisection (f: Callable[[float], float], a: float, b: float, eps: float,
              max_steps: int) -> float:

    # Lanzar un error si el intervalo dado no es válido
    if (f(a) * f(b) > 0): raise ValueError("El intervalo dado no es válido")

    # Iterar, a lo sumo, tantas veces como indique { max_steps }
    for i in range(max_steps):
        # Crear e inicializar p con un valor no válido
        p: float = np.inf

        # Usar media geométrica, si es posible
        if (a > 0 and b > 0 and b > 2 * a):
            p = np.sqrt(a * b)
        else:
            p = float((a + b) / 2)

        # Si se encontró la raíz, o la aproximación es suficientemente buena
        # , retornar p_i
        if (f(p) == 0 or (b - a) / 2 < eps):
            return p

        # Reasignar el borde según corresponda
        if (f(a) * f(p) < 0):
            b = p
        else:
            a = p

    raise Exception(f'No se logró una aproximación lo suficientemente buena
    en {max_steps} pasos.')
```

- Algoritmo de punto fijo

```
# Implementación del método de punto fijo
def fixed_point (g: Callable[[float], float], init: float, eps: float,
                max_steps: int) -> float:
    # Inicializar la variable de iteración enésima y de error, en valores
    # fuera de rango
    p: float = np.inf
    err: float = np.inf

    # Iterar hasta una cantidad de { max_steps } inclusive
    for i in range(max_steps):
        # Guardar el valor de la iteración y error actuales
        p = g(init)
        err = np.abs(p - init)

        # Verificar si el error obtenido cumple con las condiciones dadas.
        # En dicho caso, retornar el valor obtenido
        if (err < eps):
            return p

    # Almacenar el valor de iteración actual para la próxima iteración
    init = p
```

```
# Lanzar una excepción si no se logra la aproximación deseada
raise Exception(f'No se logró una aproximación lo suficientemente buena
en {max_steps} pasos.')
```